

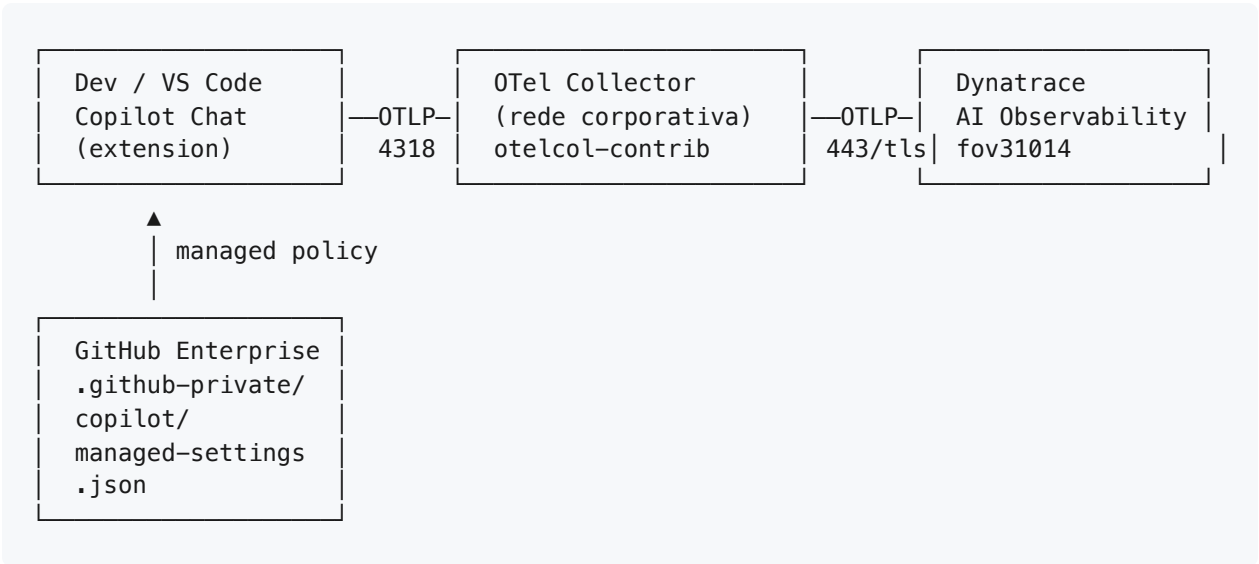
How-To — Copilot Chat Observability com Dynatrace

Guia técnico de implantação · Boa Vista · Julho 2026

Objetivo

Este documento descreve, passo a passo, como implantar a observabilidade em tempo real do GitHub Copilot Chat dos desenvolvedores da Boa Vista dentro do Dynatrace, reproduzindo os mesmos resultados obtidos na POC.

Arquitetura



Componente	Papel	Onde roda
VS Code + Copilot Chat	Emite spans/metrics OTel de cada chat, tool call e sub-agent	Máquina do dev
OTel Collector	Recebe telemetria da rede interna e forwarda para o Dynatrace (adiciona auth, opcionalmente mascara PII)	Container em VM ou Kubernetes na rede da Boa Vista
managed-settings.json	Ativa e configura OTel em todos os VS Codes automaticamente, sem que o dev toque em nada	Repositório <code>.github-private</code> na org do GHEC
Dynatrace AI Observability	Recebe, armazena no Grail, disponibiliza dashboards, app e DQL	SaaS — tenant <code>fov31014</code>

Pré-requisitos

- **GitHub Copilot Business ou Enterprise** — o plano Individual não suporta managed settings
- **GitHub Enterprise Cloud (GHEC)** com *AI Controls* habilitado, se o rollout for via server-managed. Alternativa: file-based via MDM ou Ansible/Chef/Puppet.
- **VS Code 1.128 ou superior** nos devs (para respeitar a precedência das managed policies)
- **Acesso admin ao tenant Dynatrace** para gerar API tokens
- **1 VM ou pod Kubernetes** na rede corporativa (2 vCPU / 2 GB RAM já bastam)
- **Firewall liberado:** ingresso 4318/tcp da rede de devs no collector; egresso 443/tcp do collector para `*.live.dynatrace.com`

1 Criar o token de ingest no Dynatrace

No tenant `fov31014` :

1. `Ctrl+K` → **Access Tokens** → **Generate new token**
2. Nome sugerido: `copilot-otlp-ingest`
3. Marcar os scopes:
 - `openTelemetryTrace.ingest`
 - `metrics.ingest`
4. Gerar — o token começa com `dt0c01.`
5. Salvar no cofre corporativo (1Password, Vault, AWS Secrets Manager, etc.)

Nunca distribuir esse token para as máquinas dos devs.

Ele fica

apenas no OTel Collector

. Isso permite rotação sem redeploy nos devs e reduz drasticamente a superfície de exposição.

2 Provisionar o OTel Collector

2.1 Estrutura do host

Requisitos mínimos: 2 vCPU, 2 GB RAM, 10 GB disco. Sistema operacional Linux (qualquer distribuição LTS) ou Container OS. Distros testadas: Ubuntu 22.04, Amazon Linux 2023, RHEL 9.

2.2 Arquivo `compose.yml`

```
services:
  otel-collector:
    image: otel/opentelemetry-collector-contrib:latest
    container_name: bv-otel-collector
    restart: unless-stopped
    command: ["--config=/etc/otel-collector-config.yaml"]
    volumes:
      - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml:ro
    env_file: .env
    ports:
      - "4318:4318"      # OTLP HTTP (VS Code envia aqui)
      - "4317:4317"      # OTLP gRPC (alternativa)
      - "13133:13133"    # Health check
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:13133"]
      interval: 30s
      timeout: 5s
      retries: 3
```

2.3 Arquivo `otel-collector-config.yaml`

```
receivers:
  otlp:
    protocols:
      http:
        endpoint: 0.0.0.0:4318
        max_request_body_size: 4194304
      grpc:
        endpoint: 0.0.0.0:4317

processors:
  batch:
    timeout: 5s
    send_batch_size: 512
  resource:
    attributes:
      - key: deployment.environment
        value: production
        action: upsert

exporters:
  otlphttp/dynatrace:
    endpoint: ${env:DT_OTLP_ENDPOINT}
    headers:
      Authorization: "Api-Token ${env:DT_INGEST_TOKEN}"
    compression: gzip

extensions:
  health_check:
    endpoint: 0.0.0.0:13133

service:
  extensions: [health_check]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch, resource]
      exporters: [otlphttp/dynatrace]
    metrics:
      receivers: [otlp]
      processors: [batch, resource]
      exporters: [otlphttp/dynatrace]
```

2.4 Arquivo `.env`

```
DT_OTLP_ENDPOINT=https://fov31014.live.dynatrace.com/api/v2/otlp
DT_INGEST_TOKEN=dt0c01.XXXXXXXXXXXXXXXXXXXXXXXXXX...
```

2.5 Subir o collector

```
docker compose up -d
docker logs -f bv-otel-collector
```

Validar que o health check responde:

```
curl http://otel-collector.boavista.corp:13133/
```

Teste antes do rollout

Envie um span sintético para confirmar que o pipeline funciona:

```
curl -X POST http://otel-collector.boavista.corp:4318/v1/traces \
-H "Content-Type: application/json" \
-d '{"resourceSpans": [{"scopeSpans": [{"spans": [{"
  "traceId": "00112233445566778899aabbccddeeff",
  "spanId": "0011223344556677", "name": "test",
  "startTimeUnixNano": "1721242400000000000",
  "endTimeUnixNano": "1721242401000000000"}]}]}]}'
```

Retorno **HTTP 200** → OK. Aguarde ~15s e valide no Dynatrace com DQL: `fetch spans | filter span.name == "test" | limit 5`.

3 Configurar DNS interno

Criar um registro DNS interno apontando para o host do collector:

```
otel-collector.boavista.corp → 10.x.x.x (IP do host)
```

Isso será usado no `managed-settings.json` como endpoint OTLP.

4 Publicar o `managed-settings.json` no GitHub

4.1 Selecionar organização hospedeira

1. No GitHub Enterprise Settings, ir em **AI Controls** → **Agents**
2. Selecionar a organização que hospedará o `.github-private` (geralmente uma org corporate/central)

4.2 Criar o repositório `.github-private`

Se ainda não existir, criar um repositório privado com esse nome exato na organização selecionada.

4.3 Adicionar o arquivo

Path: `copilot/managed-settings.json` (no branch default, geralmente `main`).

```
{
  "telemetry": {
    "enabled": true,
    "endpoint": "https://otel-collector.boavista.corp:4318",
    "protocol": "otlp-http",
    "captureContent": false,
    "lockCaptureContent": true,
    "serviceName": "copilot-chat-boavista",
    "resourceAttributes": {
      "deployment.environment": "production",
      "organization": "boavista",
      "cost_center": "engineering"
    }
  }
}
```

4.4 Campos explicados

Campo	Descrição
<code>enabled</code>	Liga a exportação OTEL do Copilot Chat
<code>endpoint</code>	URL do OTEL Collector interno
<code>protocol</code>	<code>otlp-http</code> (recomendado) ou <code>otlp-grpc</code>
<code>captureContent</code>	<code>false</code> = só metadados (safe LGPD). <code>true</code> = captura texto de prompt/resposta/tool. Só habilitar após aprovação do DPO.
<code>lockCaptureContent</code>	<code>true</code> impede que devs sobrescrevam <code>captureContent</code> individualmente
<code>serviceName</code>	Valor de <code>service.name</code> nos spans (default: <code>copilot-chat</code>)
<code>resourceAttributes</code>	Atributos extras adicionados a todos os spans

4.5 Alternativas de delivery

Se a Boa Vista não usa GHEC ou prefere não expor o repositório `.github-private` , existem 2 canais alternativos com precedência igual:

Canal	Path	Quando usar
File-based	Linux: <code>/etc/github-copilot/managed-setting s.json</code> macOS: <code>/Library/Application Support/GitHub bCopilot/managed-settings.json</code> Windows: <code>%ProgramFiles%\GitHubCopilot\mana ged-settings.json</code>	Deploy via Ansible/Chef/Puppet/SCCM
Native MDM	Windows Registry: <code>HKLM\SOFTWARE\Policies\Gi tHubCopilot</code>	Frotas gerenciadas por Intune, Jamf, GPO

Canal	Path	Quando usar
	macOS preference domain: <code>com.github.copilot</code>	

5 Validar em uma máquina piloto

5.1 Verificar que a policy foi aplicada

Em um VS Code de um dev cuja conta está na org GHEC configurada:

1. `Cmd/Ctrl+Shift+P` → **Developer: Policy Diagnostics**
2. Confirmar que aparece uma seção *Managed Settings* com `telemetry.enabled = true`
3. Se não aparecer imediatamente, aguardar até 1h (VS Code refaz o pull hourly)

5.2 Fazer 2-3 interações com Copilot Chat

Qualquer pergunta serve. Aguardar ~15 segundos para o batch processor do collector enviar.

5.3 Validar no Dynatrace

Notebook do tenant (`Ctrl+K` → Notebooks → novo) — colar:

```
fetch spans, from:now()-5m
| filter service.name == "copilot-chat"
| fields timestamp, span.name, gen_ai.request.model,
      gen_ai.usage.input_tokens, gen_ai.usage.output_tokens,
      user.email
| sort timestamp desc
| limit 20
```

Retorno esperado: linhas com `chat gpt-4o`, `chat claude-opus-*`, `execute_tool ...` e tokens preenchidos.

6 Enriquecimento por dev/time (opcional)

Para popular `user.email`, `user.team`, `cost_center` automaticamente em cada span:

Opção A — via `OTEL_RESOURCE_ATTRIBUTES` distribuído por MDM

Setar variável de ambiente do sistema em cada máquina via Intune/Jamf:

```
OTEL_RESOURCE_ATTRIBUTES=user.email=$USER_EMAIL,user.team=$USER_TEAM,cost_center=$COS
```

Opção B — enrichment no OTel Collector via lookup

Adicionar processor no collector que faz lookup do `user.email` em CSV/AD/LDAP e adiciona `user.team`. Requer trabalho de integração mas centraliza o mapeamento.

7 Consumir os dados no Dynatrace

7.1 App AI & LLM Observability (nativa)

Aba **Service Health** → filtrar por serviço `copilot-chat`. Aparecem automaticamente: contagem de LLM requests, tokens, latência, prompts (se `captureContent=true`), topologia de agents.

7.2 Dashboard custom (fornecido pela POC)

Importar o arquivo `dashboards/copilot-chat-observability.json` via `Ctrl+K` → Dashboards → + **Dashboard** → **Upload**. Traz widgets prontos de FinOps, SRE e adoção.

7.3 Notebook custom (fornecido pela POC)

URL do notebook criado programaticamente durante a POC:

```
https://fov31014.apps.dynatrace.com/ui/apps/dynatrace.notebooks/notebook/dcceeff0-bef
```

8 Rollback e troubleshooting

Rollback rápido

- **Server-managed:** comitar `{"telemetry":{"enabled":false}}` no `.github-private/copilot/managed-settings.json` — propaga em até 1h
- **File-based:** remover o arquivo via Ansible
- **Native MDM:** remover a policy da MDM

VS Code não emite nada

1. `Cmd/Ctrl+Shift+P` → **Developer: Policy Diagnostics** — confirmar que `telemetry.enabled` está true
2. Testar exporter local: setar `github.copilot.chat.otel.exporterType` para `"console"` — spans aparecem no Output panel do VS Code (canal "GitHub Copilot Chat")
3. Verificar conectividade: `curl -v http://otel-collector.boavista.corp:4318/v1/traces -H "Content-Type: application/json" -d "{}"`

Collector recebe, Dynatrace não mostra

- Ver logs do collector: procurar HTTP 401 (token inválido), 403 (scope faltando), erro de DNS
- Confirmar que endpoint é `*.live.dynatrace.com` e não `*.apps.dynatrace.com`

Considerações de LGPD

Modo padrão (`captureContent = false`):

apenas metadados. Zero risco de PII. Não requer aprovação do DPO.

Modo `captureContent = true`:

captura texto de prompt/resposta/tool. Inclui código-fonte, dados de arquivos abertos, comandos de terminal.

- Aprovação do DPO necessária
- Comunicação transparente aos devs
- Habilitar redação de PII no collector (regex CPF, email, tokens)
- Alinhar retenção no Grail (default 35d)

Referências oficiais

- VS Code — Monitor agent usage with OpenTelemetry — code.visualstudio.com/docs/agents/guides/monitoring-agents
- VS Code — Manage AI settings in enterprise — code.visualstudio.com/docs/enterprise/ai-settings
- GitHub — Enterprise-managed OpenTelemetry export (jul/2026) — github.blog/changelog/2026-07-08
- GitHub — Enterprise managed-settings.json GA (jul/2026) — github.blog/changelog/2026-07-01
- Dynatrace — AI & LLM Observability — docs.dynatrace.com/docs/observe/dynatrace-for-ai-observability
- OTel — GenAI Semantic Conventions — github.com/open-telemetry/semantic-conventions/blob/main/docs/gen-ai